

Java 21

Java 21 introduction

Di cosa parla: è la lezione di apertura. Serve a inquadrare *cosa* si studierà e *perché*.

Punti chiave:

- **Java 21 è una release LTS** (Long-Term Support), cioè con supporto a lungo termine. La precedente LTS era **Java 17** (quella che hai studiato con sealed, record, switch, pattern matching, text block).
- Un **JEP** è un *JDK Enhancement Proposal*, cioè una proposta ufficiale di miglioramento del linguaggio. Ogni funzionalità nuova nasce da un JEP.
- Stato delle funzionalità in Java 21:
 - **Switch Pattern Matching** → era in *preview* da Java 17, ora è **finalizzato**.
 - **Record Patterns** → più *preview*, ora **finalizzati**.
 - **Unnamed Classes + Instance Main Methods** → ancora in **preview** (JEP 445).
 - **Sequenced Collections** → arrivate **direttamente finalizzate**, senza *preview*.
 - **Virtual Threads** → *preview* da Java 19, ora **finalizzati**.

Poiché al momento della registrazione Java 21 non era ancora uscito ufficialmente, il docente ha scaricato una **build anticipata da OpenJDK** e compila da riga di comando con `javac`, senza IDE (che non supportava ancora la versione 21).

IDE and javac

Di cosa parla: come compilare ed eseguire codice Java 21 **da riga di comando**, dato che l'IDE (IntelliJ 2020.3.2) non supportava ancora tutte le feature.

Concetti importanti:

- Alcune feature Java 21 (es. **costanti enum qualificate**, unnamed classes) **non compilano** in un IDE fermo alla versione 20. Quindi si usa il compilatore ufficiale **javac**.
- Per usare `javac` da qualunque cartella, va aggiornata la **variabile d'ambiente PATH** puntando alla cartella `bin` del JDK 21 (serve un privilegio di amministratore). Così il comando viene "trovato" automaticamente.

I due flag fondamentali:

```
# Compilazione: -d dice DOVE mettere i file .class generati
javac -d target sorgente/MioFile.java

# Da "target" javac ricostruisce le sottocartelle in base al package
# Esecuzione: -cp (classpath) dice DOVE cercare i .class
java -cp target nomepackage.MioFile
```

Mnemonica utile: -d = *destination* (dove **scrivo** i .class in compilazione); -cp = *classpath* (dove **leggo** i .class in esecuzione).

Unnamed Classes and Instance Main Methods (*preview*)

Definizione: è la feature (JEP 445) che permette di scrivere un programma Java **senza dichiarare la classe** e con un metodo **main semplificato**, cioè un *instance main method* invece del tradizionale `public static void main(String[] args)`.

Perché esiste (motivazione): Java supporta due tipi di programmazione:

- *programming in the small* → variabili, metodi, controllo di flusso;
- *programming in the large* → classi, interfacce, package, moduli.

L'obiettivo è **ridurre la "cerimonia"** per chi impara Java le prime settimane: costrutti come `class`, il modificatore `public`, la parola chiave `static` appartengono al "programming in the large" e non dovrebbero servire subito. Si vuole rendere Java **più facile da imparare**.

Come si semplifica, passo per passo:

// *PRIMA (classico)*

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello world");  
    }  
}
```

// *DOPO (instance main method + unnamed class)*

```
void main() {  
    System.out.println("Hello world");  
}
```

Si tolgono nell'ordine: il parametro `String[] args`, poi `static` e `public` (→ *instance main method*), infine la dichiarazione `class` (→ *unnamed class*).

Regole delle unnamed classes (importanti):

- Estendono implicitamente `Object` e **non possono implementare interfacce**.
- Sono **final** e risiedono nel **pacchetto senza nome** (unnamed package).
- Il nome del file `.class` sul disco dipende dal **nome del file sorgente**:
`HelloWorld.java` → genera `HelloWorld.class`.
- Hanno **un solo costruttore**: quello di default no-args fornito dal compilatore. **Scrivere esplicitamente un costruttore è un errore**.

- La parola chiave `this` resta valida, ma **non puoi istanziare** una unnamed class per nome dall'esterno (il codice non può riferirla per nome). Quindi servono solo come **punto di ingresso** di un programma → devono avere un `main`.
- Se coesistono *sia* il `main` tradizionale *sia* l'instance `main`, **vince quello tradizionale** (`public static void main(String[])` ha la precedenza).

Compilazione (feature preview → va abilitata esplicitamente):

```
javac --release 21 --enable-preview HelloWorld.java
java --enable-preview HelloWorld
```

Con `javap` (il *disassembler*) si verifica che dietro le quinte la classe generata è `final`, estende `Object` e ha solo il costruttore di default.

Unnamed Classes and Instance Main Methods (Code)

Di cosa parla: la versione "live" della lezione precedente, con dimostrazioni pratiche in Notepad++ e `javac`. Serve a *vedere accadere* le regole appena elencate.

Cosa dimostra:

1. **Variabile e metodo di istanza** dentro una unnamed class:

```
int x = 5;
void showX() { System.out.println(x); } // metodo di istanza
void main() { showX(); }                // stampa 5
```

2. **Errore se scrivi un costruttore:** aggiungendo anche solo un costruttore no-args, `javac` dà errore ("*expected class, interface, enum or record*") → conferma che il costruttore lo mette **solo** il compilatore.
3. **Non puoi istanziare la unnamed class** (`new HelloWorld()`): pur esistendo il file `HelloWorld.class` sul disco, il compilatore dà "*cannot find symbol*", perché non è riferibile per nome.
4. **Classi extra nello stesso file** diventano **inner class**: si vede sul disco il file `HelloWorld$AltraClasse.class`. Il `$` è la convenzione di naming che separa classe **esterna** e classe **interna**.

Morale didattica del docente: con i principianti **non** mescoleresti classi extra; le terrestri come classi separate nel pacchetto di default. Il vantaggio è **mantenere il "programming in the small"** finché non serve davvero introdurre i costrutti "grandi" (`public`, `static`, `class`, array di stringhe).

Record Patterns

Definizione: un **record pattern** permette, in un `instanceof` o in uno `switch`, di **verificare** che un oggetto sia di un certo tipo record e **contemporaneamente destrutturarlo** (estrarre i suoi componenti in variabili locali) in un colpo solo.

Prima però la lezione ripassa due basi:

a) Type Pattern (da Java 16): instanceof esteso per fare pattern matching, eliminando il vecchio idiomma cast.

```
// VECCHIO (pre-Java 16): controllo + cast espliciti
```

```
if (obj instanceof String) {  
    String s = (String) obj;    // cast manuale  
    System.out.println(s.toUpperCase());  
}
```

```
// NUOVO: type pattern "String s"
```

```
if (obj instanceof String s) { // se vero, s È già la stringa  
    System.out.println(s.toUpperCase());  
}
```

Lo stile è più **dichiarativo**: dici cosa vuoi, non *come* ottenerlo. Il matching avviene **a runtime**: se il pattern combacia, l'espressione è `true` e `s` fa riferimento all'oggetto.

b) Ripasso record (già visti nel tuo corso Java 17): un record è una classe speciale, **immutabile**, "data carrier". Dichiarando i **componenti** ottieni automaticamente: campi `private final`, **costruttore canonico**, metodi accessori con lo **stesso nome** del componente, più `toString()`, `equals()`, `hashCode()`.

```
public record Dog(String name, int age) {}
```

```
// accessor: name() e age() (NON getName())
```

c) Record Pattern vero e proprio: un record pattern fa **due cose**:

1. controlla se l'oggetto supera l'`instanceof`;
2. **destruttura** il record nei suoi componenti, popolando variabili locali chiamando gli accessor **per te**.

```
public record Person(String name, int age) {}
```

```
// Con SOLO type pattern devi chiamare tu gli accessor:
```

```
if (obj instanceof Person p) {  
    String s = p.name();  
    int n = p.age();  
}
```

```
// Con RECORD PATTERN la destrutturazione è automatica:
```

```
if (obj instanceof Person(String s, Integer age)) {  
    // s e age sono già popolati chiamando name() e age()  
}
```

Attenzione (edge case): i nomi delle variabili locali (s, age) possono essere **diversi** dai componenti; ma **l'ordine e i tipi** dei pattern componente **devono corrispondere** all'ordine dei componenti del record. Supporta anche la **nidificazione** (record dentro record).

Record Patterns (in code)

Di cosa parla: dimostrazione pratica dei record pattern, inclusa la **nidificazione** e i record **generici**.

Setup dei record (con annidamento crescente):

```
record Person(String name, int age) {}  
enum Ability { WEAK, MEDIUM, STRONG }  
record Player(Person person, Ability ability) {} // annida Person  
record Doubles(Player player1, Player player2) {} // annida due Player  
record Pair<X, Y>(X x, Y y) {} // record GENERICO
```

Il "grafico dell'oggetto": un `Doubles` contiene due `Player`, ognuno contiene una `Person` e una `Ability`. Livelli di annidamento.

Record pattern semplice:

```
// Se true, s = person.name() e age = person.age(), automaticamente  
if (obj instanceof Person(String s, Integer age)) {  
    System.out.println(s + " " + age); // es. "Joe Bloggs 22"  
}
```

Record pattern con generics: passando un `Pair<String, String>` matcha il pattern per stringhe; passando un `Pair<Integer, Integer>` matcha quello per interi (destrutturando x e y).

Record pattern nidificato (il cuore della lezione): puoi **scendere** oltre il primo livello e destrutturare anche il record annidato.

```
// Nidificazione "semplice": mi fermo a Player  
if (obj instanceof Doubles(Player pl1, Player pl2)) {  
    pl1.person(); pl1.ability(); // uso gli accessor di Player  
}
```

```
// Nidificazione "profonda": scompongo Player in Person + Ability
```

```

if (obj instanceof Doubles(Player(Person p1, Ability a1), Player pl2)) {
    System.out.println(p1.name() + " " + p1.age()); // arrivo dritto alla Person
    System.out.println(a1.name());
}

```

Nota finale del docente: funziona anche con **var** (LVTI) al posto dei tipi espliciti, ma è più chiaro all'inizio scrivere i tipi. **Vantaggio complessivo:** niente più cast espliciti e niente più chiamate manuali agli accessor — Java popola le variabili locali per te.

Pattern Matching for Switch

Definizione: estende `switch` (statement ed expression) per usare i **pattern** nelle etichette `case`, invece dei soli valori costanti. Introdotto in preview da Java 17, **finalizzato in Java 21**.

Motivazione: lo `switch` si presta naturalmente al pattern matching, e permette di **separare** la logica di selezione (a sinistra: etichette, pattern, guardie) dalla **business logic** (a destra della freccia `->`).

Novità principali:

1. **Le etichette `case` possono contenere pattern e `null`.** Prima `null` causava una `NullPointerException`; ora puoi gestirlo con un `case null`.
2. **Clausole `when` (le "guardie"):** una **guardia** è un'espressione booleana a destra di `when`, per raffinare il `case`. (Introdotte in Java 19; sostituiscono i vecchi "guarded patterns" della preview di Java 17.)

Vehicle v = ...;

```

String result = switch (v) {           // v = selector expression
    case Boat b -> "è una barca";     // type pattern label
    case Train t -> "è un treno";
    case Car c when c.numDoors() == 4 -> "berlina"; // guardia
    case Car c when c.numDoors() == 2 -> { // blocco con yield
        yield "coupé";                // yield in una switch expression
    }
    default -> "altro veicolo";        // cattura tutti gli altri casi
};

```

3. **Selector expression estesa a qualsiasi reference type.** Prima potevi `switchare` solo su: primitivi integrali (escluso `long`), i relativi wrapper, `String` ed `enum`. Ora puoi `switchare` su **qualsiasi tipo di riferimento** (record, array, `Object`, ecc.).

Object obj = ...;

```

String r = switch (obj) {

```

```

case String s -> "String";
case Record rec -> "Record";
case MyEnum e -> "Enum";
case null -> "null";
case double[] d -> "array di double";
default -> "default"; // es. per un Integer
};

```

4. **Costanti enum qualificate** ora ammesse nei case (es. `Primary.RED` invece del solo `RED`). Utile con **interfacce sigillate** implementate da più enum, per rendere il codice **meno prolisso** rispetto a `when x == Primary.RED`.

Due proprietà **da ricordare bene** (spesso trabocchetti d'esame):

- **Label dominance (dominanza delle etichette):** analoga alle clausole `catch` — un'etichetta più generale "domina" quelle più specifiche che la seguono, rendendole **irraggiungibili**. Es: `case Vehicle v` prima di `case Boat b` → `Boat` è irraggiungibile (`Boat` è sottotipo di `Vehicle`) → **errore di compilazione**. Idem: un pattern **incondizionato** (senza `when`) + un `default` → il compilatore segnala che il `default` è dominato.
- **Exhaustiveness (esaustività):** vale sia per switch expression sia per statement con pattern. Lo switch **deve coprire tutti i possibili valori** del selettore. Se switchi su `Object` e gestisci solo `String`, `Integer`, `null`, mancano infiniti altri tipi (`StringBuilder`, `Thread`...) → serve un **default**. Errore tipico: *"switch expression does not cover all possible input values"*.

Pattern Matching for Switch (code)

Di cosa parla: dimostrazione pratica dei concetti sopra. Alcuni esempi girano già in Java 20 (nell'IDE), altri richiedono `javac` con Java 21.

Cosa mostra concretamente:

a) switch statement vs switch expression:

- Un'expression restituisce sempre un valore (es. `return switch(...)` o assegnazione).
- Uno statement esegue azioni (es. `System.out.println` dentro i case) senza restituire un valore.

b) **case null** per non incappare nella `NullPointerException`.

c) Guardie con when:

```

switch (obj) {
  case String s when s.startsWith("A") -> "stringa che inizia con A";

```

```

case Integer i when i > 10      -> "intero maggiore di 10";
case null                      -> "null";
default                        -> "non riconosciuto";
}

// "abc" (minuscolo) → default; "12" → "intero > 10"

```

A **sinistra** della freccia: etichette + guardie. A **destra**: business logic.

d) Esaustività dal vivo: togliendo il case `Vehicle/default`, compare l'errore *"switch does not cover all possible input values"*. Passando un `new Vehicle()` senza un case adatto → errore.

e) Dominanza dal vivo:

- case `Vehicle` + default insieme → **unconditional pattern** che domina il default.
- default + null → il default domina il null (irraggiungibile).
- Soluzione elegante introdotta: l'etichetta combinata **case null, default** -> (stessa etichetta per null e default).

f) Costanti enum qualificate: l'esempio con interfaccia sigillata `Color` implementata dagli enum `Primary` (`RED`, `GREEN`, `BLUE`) e `Rainbow` (`RED`, `ORANGE`, ...). Switchando sull'**interfaccia** `Color` (non sul singolo enum), le guardie verbose `when x == Primary.RED` si sostituiscono con l'etichetta **case Primary.RED**. Ma questo **compila solo con Java 21** (l'IDE fermo a Java 20 dà: *"an enum switch case label must be the unqualified name of an enumeration constant"*), quindi va compilato con `javac --release 21`.

Sequenced Collections

Definizione: nuove interfacce (JEP 431) che danno un **ordine di incontro (encounter order)** **definito e coerente** alle collezioni, con API uniformi per accedere al **primo** e all'**ultimo** elemento e per scorrere in **ordine inverso**.

Motivazione — il problema che risolvono: il Collections Framework era **incoerente** riguardo all'ordine:

- `List` e `Deque` definiscono un ordine, ma il loro supertipo comune `Collection` no.
- `Set` non definisce ordine, `HashSet` neppure, ma `LinkedHashSet` e `TreeSet` (`SortedSet`) sì — **senza però un'interfaccia comune** che lo dichiarasse.
- Ottenere il **primo/ultimo** elemento si faceva in modi tutti **diversi**:
 - `List` → `get(0)` e `get(size()-1)`;
 - `Deque` → `getFirst()` / `getLast()`;
 - `SortedSet` → `first()` / `last()`;
 - `LinkedHashSet` → dovevi prendere un iteratore e fare `next()`, e per l'ultimo **non c'era** alcun metodo.
- **Iterare al contrario** era ancora più disomogeneo (`NavigableSet.descendingSet()`, `Deque.descendingIterator()`, ecc.), e con `LinkedHashSet` dovevi **copiare** in un'altra collezione.

La soluzione — tre nuove interfacce:

SequencedCollection **extends** Collection

SequencedSet **extends** Set, SequencedCollection

SequencedMap **extends** Map

Nella gerarchia, `List` diventa sotto-interfaccia di `SequencedCollection`; le uniche **classi** annotate come implementazioni nuove sono `LinkedHashSet` e `LinkedHashMap`.

Metodi nuovi (tutti con implementazioni di default):

SequencedCollection:

```
SequencedCollection<E> reversed(); // vista in ordine inverso
```

```
void addFirst(E); void addLast(E);
```

```
E getFirst(); E getLast();
```

```
E removeFirst(); E removeLast();
```

`SequencedSet`: come sopra ma `reversed()` restituisce un `SequencedSet` (niente duplicati).

SequencedMap:

```
SequencedMap<K,V> reversed();
```

```
SequencedSet<K> sequencedKeySet();
```

```
SequencedCollection<V> sequencedValues();
```

```
SequencedSet<Map.Entry<K,V>> sequencedEntrySet();
```

```
void putFirst(K,V); void putLast(K,V);
```

```
// + firstEntry(), lastEntry(), pollFirstEntry(), pollLastEntry()
```

In una frase: ora hai **un'unica API coerente** per lavorare su testa/coda e per scorrere avanti/indietro, valida su liste, set e mappe.

Java 21 – Sequenced Collections (Code)

Di cosa parla: le tre interfacce in azione. Il docente sfrutta anche le **unnamed classes + instance main** (lezioni 3–4) perché rendono l'esempio velocissimo da scrivere.

SequencedCollection (con `ArrayList`, che **ammette duplicati**):

```
SequencedCollection<String> sc = new ArrayList<>();
```

```
sc.addFirst("A"); // [A]
```

```
sc.addFirst("B"); // [B, A]
```

```
sc.addLast("C"); // [B, A, C]
```

```
sc.addLast("D"); // [B, A, C, D]
sc.addLast("D"); // [B, A, C, D, D] -> duplicato ammesso
```

```
sc.getFirst(); // B
sc.getLast(); // D
sc.removeFirst(); // rimuove B -> [A, C, D, D]
sc.removeLast(); // rimuove D -> [A, C, D]
```

```
for (String s : sc) { ... } // avanti: A, C, D
for (String s : sc.reversed()) {} // indietro: D, C, A
```

SequencedSet (con `LinkedHashSet`, che **NON ammette duplicati**): stessi identici `add`, ma la seconda **"D"** non viene aggiunta.

```
SequencedSet ss = new LinkedHashSet<>(); // stessi add: A,B,C,D,D -> risulta [B, A, C, D] //
dopo removeFirst/removeLast restano A e C // avanti: A, C | ss.reversed(): C, A
```

SequencedMap (con `LinkedHashMap`): API "gemella" ma con chiave+valore.

```
SequencedMap<Integer,String> sm = new LinkedHashMap<>();
sm.putFirst(1, "Adam"); // {1=Adam}
sm.putFirst(2, "Brian"); // {2=Brian, 1=Adam}
sm.putLast(3, "Charlie"); // {2=Brian, 1=Adam, 3=Charlie}
sm.putLast(4, "David"); // {..., 4=David}
```

```
sm.firstEntry(); // 2=Brian
sm.lastEntry(); // 4=David
sm.pollFirstEntry(); // restituisce 2=Brian E lo rimuove
sm.pollLastEntry(); // restituisce 4=David E lo rimuove
// restano {1=Adam, 3=Charlie}
```

```
// avanti: sequencedEntrySet() -> Adam, Charlie
// indietro: reversed() -> Charlie, Adam
```

Nota di compilazione: le Sequenced Collections **non sono preview**, ma poiché l'esempio usa le **unnamed classes** (che lo sono), la compilazione richiede comunque `--release 21 --enable-preview`.

Morale: l'enorme vantaggio è la **coerenza** — stessi metodi per operare su testa/coda e, soprattutto, scorrere al contrario è finalmente **semplice** (`reversed()`).

11. Virtual Threads

Definizione: i **thread virtuali** sono thread **leggeri, poco costosi e molto abbondanti**, gestiti dalla **JVM** (non dal sistema operativo), pensati per far **scalare** le applicazioni server scritte nello stile *thread-per-request*. Preview da Java 19, **finalizzati in Java 21**.

Motivazione — il problema: i thread tradizionali (**platform threads**) sono un sottile involucro attorno ai **thread del sistema operativo (OS threads)**, che sono **costosi**. Se ogni richiesta consuma un OS thread per tutta la sua durata, il numero di thread diventa il **collo di bottiglia** nello scaling — e il *thread pooling* non basta, perché non aumenta il numero totale di thread.

L'idea (analogia): come il SO dà l'illusione di memoria abbondante mappando un ampio spazio virtuale su poca RAM fisica, così il runtime Java dà l'**illusione di thread abbondanti** mappando **tanti thread virtuali su pochi OS thread**. Si **rompe la relazione 1:1** tra thread Java e thread OS.

Virtual thread vs Platform thread:

	Platform thread	Virtual thread
Mapping	1:1 con un OS thread	molti VT multiplexati su pochi OS thread
Risorse	pesante, stack grande, molta memoria	leggero, stack piccolo, memoria minima
Scheduling/ blocco	gestito dall'OS; il blocco occupa l'OS thread	gestito dalla JVM; quando si blocca, libera l'OS thread
Scalabilità	limitata (migliaia sono impraticabili)	altissima (decine di migliaia → milioni)

Concetti chiave:

- Un virtual thread **consuma un OS thread solo mentre calcola sulla CPU**. Quando esegue un'operazione **bloccante** (I/O, database) viene **sospeso** automaticamente e **smontato** dall'OS thread, che torna libero.
- I VT **non sono più veloci** dei platform thread: esistono per dare **scala, non velocità**. L'obiettivo è più **throughput** (richieste/secondo), non minore latenza.
- Vantaggio massimo quando: **molti task concorrenti** (più di qualche migliaio) e carico **non CPU-bound** (i task passano molto tempo **in attesa**).
- **Non si mettono in pool:** essendo economici, si usa un executor **un-VT-per-task** invece di un pool.

Scheduling — carrier threads: i platform thread usano direttamente lo scheduler dell'OS. I virtual thread usano lo **scheduler del JDK**, che li **monta** su platform thread detti **carrier**; i carrier sono poi schedulati dall'OS. Un VT può essere **montato/smontato** su carrier diversi durante la sua vita: tipicamente si smonta quando si blocca (I/O, DB) e si rimonta su un carrier disponibile a operazione conclusa. Il tutto avviene **frequentemente e in modo trasparente**, senza bloccare OS thread.

L'esempio di codice confronta 3 executor sullo stesso lavoro (N task, ognuno "dorme" 1 secondo per simulare una chiamata bloccante):

// try-with-resources: l'executor si chiude solo dopo che TUTTI i task finiscono

```
try (var executor = /* uno dei tre qui sotto */) {  
    IntStream.range(0, NUM_TASKS).forEach(i ->  
        executor.submit(() -> { Thread.sleep(1000); return i; })  
    );  
}
```

Risultati mostrati:

- **Executors.newCachedThreadPool()** (un platform thread per task): ~1000 task OK (~1 sec), ma a **10.000 task** → **OutOfMemoryError**. Non scala.
- **Executors.newFixedThreadPool(200)** (max 200 platform thread): con 10.000 task e 1 sec ciascuno → throughput 200 task/sec → serve **~50 secondi**. Funziona ma lento.
- **Executors.newVirtualThreadPerTaskExecutor()**: 10.000 task in **~1 sec**; 100.000 veloce; **1 milione** rapido; **2 milioni** in **~8 secondi**. Enorme numero di operazioni montaggio/smontaggio di tanti VT su pochi carrier → **scalabilità eccellente**.

Con questa lezione si conclude la sezione Java 21 del corso.